

DATA REPRESENTATION AND SERIALIZATION FORMATS

Integração de Sistemas

Daniel Pereira
Eduardo Marques

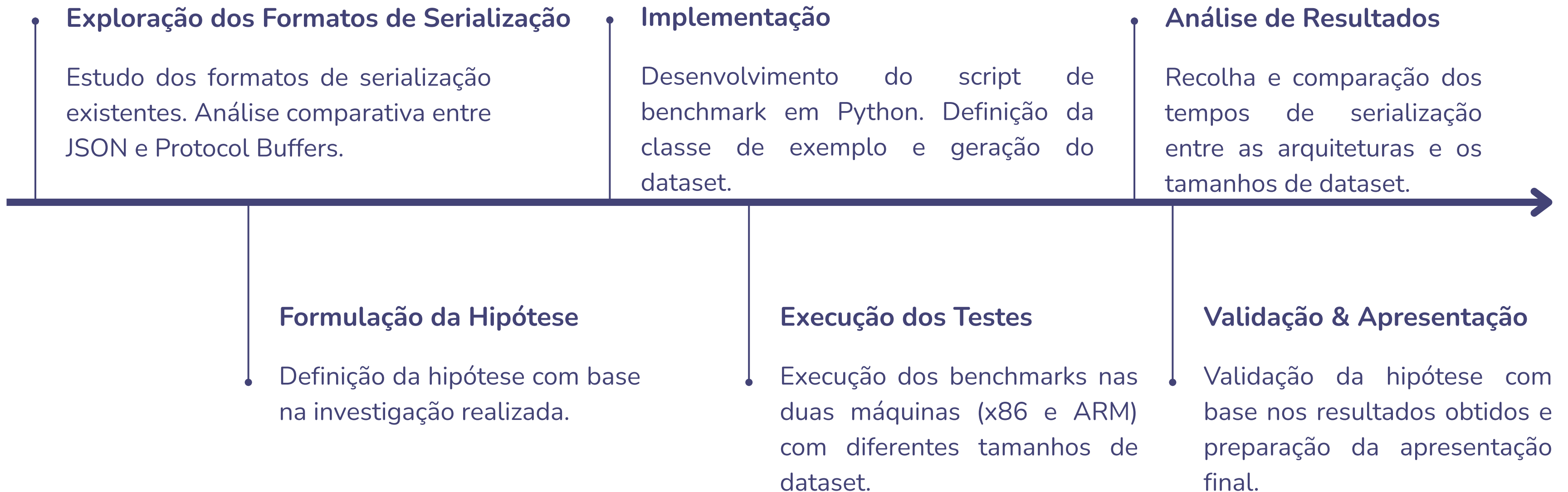
CONTEXTUALIZAÇÃO

O que é a serialização? Processo de converter objetos em memória num formato transmissível ou armazenável, permitindo a comunicação entre sistemas.

Por que é relevante? Em sistemas modernos, a troca de dados entre serviços é constante — a eficiência do formato de serialização impacta diretamente a performance, escalabilidade e consumo de recursos.

Objetivo — Comparar o desempenho de serialização entre JSON e Protocol Buffers em diferentes volumes de dados e arquiteturas de processador.

WORKFLOW



HIPÓTESE

Protocol Buffers apresenta maior eficiência do que JSON em termos de tempo de serialização, sendo essa vantagem mais evidente em datasets maiores. Os resultados obtidos podem variar consoante a arquitetura do processador utilizado.

COMPARAÇÃO DOS FORMATOS

JSON

Legibilidade

Formato de texto legível por humanos.

Tamanho dos Dados

Estrutura textual com chaves repetidas.

Compatibilidade

Suportado nativamente por browsers e APIs REST.

Flexibilidade de Schema

Schema opcional — permite estruturas dinâmicas e alterações sem recompilação.

Protobuf

Serialização

Formato binário compacto.

Performance

Serializações mais rápidas, ideal para alta carga.

Esquema rigoroso

Schema obrigatório — reduz erros e garante consistência.

Casos de Uso

Ideal para comunicação entre microserviços e sistemas com requisitos de baixa latência.

BENCHMARKING SETUP

Máquina	Processador	Arquitetura	Núcleos	RAM	SO
MacBook	Apple M1	ARM	8	8 GB	macOS Tahoe 26.3
MSI	12th Gen Intel(R) Core(TM) i7-1280P (2.00 GHz)	x86	14	16 GB	Windows 11 Home

Tabela 1 - Especificações das Máquinas utilizadas

Para garantir uma comparação justa, foram utilizadas as mesmas versões do Python (3.10.12) e de todas as bibliotecas em ambas as máquinas.

CONFIGURAÇÃO DA EXPERIÊNCIA

Métodos de Serialização:

- JSON → `json.dumps()`
- Protobuf → `SerializeToString()`

Tamanho do dataset:

- 10_000
- 100_000
- 1_000_000
- 5_000_000

Número de iterações:

- Warmup: 10
- Benchmark: 40

Estrutura da Classe:

- `user_id` → string aleatória
- `age` → inteiro (18–90)
- `account_balance` → float
- `is_premium` → boolean
- `status` → enum

Garantia de Reprodutibilidade:

RANDOM_SEED = 42 assegura que os dados gerados são sempre os mesmos em qualquer execução.

RESULTADOS OBTIDOS

JSON VS PROTOBUF

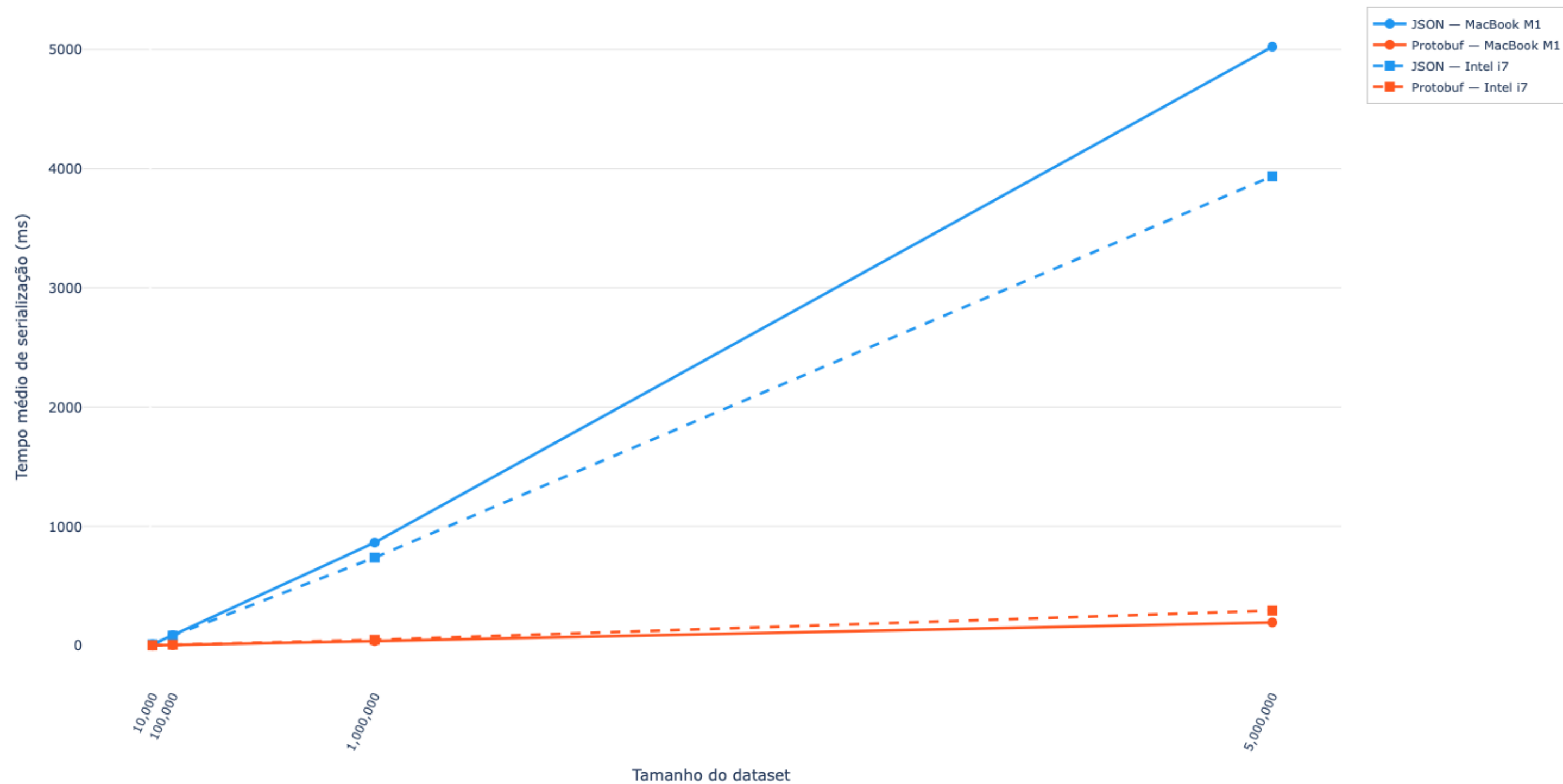


Gráfico 1 - Serialization Benchmark · Tempo médio (ms)

RESULTADOS OBTIDOS

JSON VS PROTOBUF

● JSON ● Protobuf

TAMANHO	JSON X86	JSON ARM	PROTOBUF X86	PROTOBUF ARM
10 000	7,398 ms	8,795 ms	0,663 ms 11×	0,370 ms 24×
100 000	82,087 ms	85,514 ms	5,373 ms 15×	3,603 ms 24×
1 000 000	736,776 ms	863,129 ms	46,333 ms 16×	35,998 ms 24×
5 000 000	3935,993 ms	5022,448 ms	291,721 ms 13×	192,600 ms 26×

Tabela 2 - Serialization Benchmark · Tempo médio (ms)

CONCLUSÕES

✓ **Hipótese confirmada** — o Protobuf foi consistentemente mais rápido que o JSON em todos os tamanhos do dataset e em ambas as arquiteturas.

A vantagem aumenta com o dataset — os rácios de desempenho mantêm-se estáveis ou crescem à medida que o dataset aumenta, tornando o Protobuf especialmente vantajoso em grandes volumes de dados.

Influência da arquitetura:

- O x86 (MSI) apresentou tempos de serialização JSON significativamente mais baixos, revelando melhor desempenho absoluto nesse formato.
- O ARM (MacBook M1) destacou-se no Protobuf, com rácios de vantagem superiores (~24–26× face a ~11–16× no x86).



Para sistemas que processam grandes volumes de dados, o Protobuf torna-se uma escolha claramente melhor em relação ao JSON em termos de eficiência de serialização, independentemente da arquitetura utilizada.

REFERÊNCIAS

<https://beeceptor.com/docs/concepts/data-exchange-formats/>

<https://www.json.org/json-en.html>

<https://protobuf.dev>

Ferramentas de IA utilizadas:

- ChatGPT (OpenAI)
- Claude (Anthropic)

FIM

Obrigado pela atenção!

Daniel Pereira
Eduardo Marques